Voice Agent — speech pipeline

How the Voice Agent (Clasificar → Voice Agent) hears the worker, decides what they meant, and what is needed to build, run and ship it.

Last updated: 7 October 2026.

Status at a glance

Part	State
Live mic session streaming to Deepgram	Built. Working on a test phone (Redmi Note 11, Android 13), English.
Offline fallback (Vosk)	Unchanged, still the fallback when there is no connection.
`interpret` Cloud Function on Claude	Built and tested locally. **Not deployed.**
`transcribe` Cloud Function on Deepgram	Built and tested locally. **Not deployed.** No longer used while the live session is connected.
`deepgramToken` Cloud Function	Built. **Cannot work with the current Deepgram key** (see Known issues).
Jev (TypeSafe) confidence scoring for option picks	**Part of the architecture, not built yet.** Measured from a computer: 24 of 25 test answers right, about 0.1 s each. See Option scoring with Jev.

Nothing in `functions/` has been deployed from this checkout. Until it is, the app on a phone has live transcription (through the development key, below) but no cloud interpretation: an answer the on-device parser cannot read falls back to "say the number or the name".

The architecture

Five stages. Each one has a single job, and the cheaper, faster stage always runs first.

#	Stage	Tool	Job	Needs network	State
1	Hear	Deepgram `nova-3`, live stream	Speech → text	Yes	Built
1b	Hear, offline	Vosk	Speech → text on the phone	No	Built
2	Read exact answers	On-device parser	Numbers, weights, yes/no, "go back", option numbers	No	Built
3	Pick an option	**Jev** (TypeSafe `choice`)	Text → the closest option, with a confidence score	Yes	**Not built yet**
4	Understand commands	Claude Opus 5.5 (`interpret`)	Summary commands, messy corrections	Yes	Built, not deployed

#	Stage	Tool	Job	Needs network	State
5	Speak	Device text-to-speech	Monte's voice	No	Built

```
Worker speaks
   │
   ▼
Phone microphone ── echo canceller removes most of Monte's own voice
   │
   ▼
1  StreamSession (native) ── 16 kHz PCM, 20 ms frames ──►  Deepgram WebSocket (nova-3)
   │                                                              │
   │  ◄──────────────── interim / final / speech_final ──────────┘
   ▼
   useSpeech.js      shows interim text live, commits one complete answer
   │
   ▼
   VoiceAgentFlow.js handleUtterance
   │
   ├─ 2  a number, a weight, yes / no, "go back"
   │        └─► on-device parser ──► act                          (no network)
   │
   ├─ 3  an answer on an option screen
   │     (client, location, category, sub-category)
   │        └─► Jev choice ──► option + confidence                (NOT BUILT YET)
   │               ├─ 0.9 or more ──► select
   │               ├─ 0.5 to 0.9 ──► select, ask the worker to confirm
   │               └─ below 0.5, or "none of these" ──► ask again
   │
   └─ 4  anything else the parser cannot read
            └─► Claude `interpret` ──► one canonical command ──► act
                  └─ unclear ──► "say the number or the name"
   │
   ▼
5  Monte speaks the next question, the mic keeps listening
```

Today, stage 3 does not exist. Option answers go to the on-device parser and, when it cannot read them, to Claude. Everything else in the diagram is built.

With no connection, stage 1 runs on Vosk turn by turn (Monte speaks, the mic opens for one answer), stage 2 handles every answer, and stages 3 and 4 are skipped: an unreadable answer gets "say the number or the name".

The live mic session

When it runs

One session per collection. It opens after the worker picks a language and the client list has loaded, and stays open through client → location → category → sub-category → weight → summary. It closes when:

- the collection is submitted (the `done` screen),
- the worker taps Exit or leaves the screen,
- voice is switched off (🔇),
- the Mic test screen is opened (it uses the microphone itself).

This is set in `VoiceAgentFlow.js` where `useSpeech` is called:

```
live: {
    enabled: voiceOn && !showTest && phase !== 'loadingctx' && phase !== 'done',
    keyterms: liveKeyterms,
},
```

Deepgram settings

Built in `components/voice/deepgramStream.js` (`liveUrl`).

Parameter	Value	Why
`model`	`nova-3`	
`language`	`es` or `en`	The language the worker picked.
`encoding`, `sample_rate`, `channels`	`linear16`, `16000`, `1`	What the native recorder produces.
`interim_results`	`true`	Live text while the worker is still talking.
`endpointing`	`500`	Milliseconds of silence that end an answer.
`utterance_end_ms`	`1000`	Backstop for when background noise hides that silence.
`smart_format`	`true`	"veinticuatro kilos" → "24 kilos". **Has a known bug**, see below.
`keyterm` (repeated)	up to 60	Names the worker is likely to say: "kilos", "Monte", category and sub-category labels, client names, location names. Fixed when the session connects.

When an answer is committed

Deepgram sends three kinds of result. Only a complete answer reaches the flow.

Message	Meaning	What the app does
`is_final: false`	Still improving this stretch of speech	Shows it as live text.
`is_final: true`	This stretch is settled	Adds it to the answer so far.

Message	Meaning	What the app does
`speech_final: true`	The worker stopped talking	Commits the whole answer to `onTranscript`.
`UtteranceEnd`	Backstop end-of-speech signal	Commits whatever has been collected.

Before committing, `cleanTranscript` strips punctuation and keeps decimal points: `"Plástico, 24,5 kilos."` → `"Plástico 24.5 kilos"`.

Not transcribing Monte

The mic records on the phone's call-audio path with the echo canceller on, and Monte speaks on that same path, so most of Monte's voice is removed. What is left would still be transcribed as if the worker had said it, so the native session stops sending audio while Monte talks.

State	When	Audio sent to Deepgram
`OPEN`	The worker's turn	Every frame
`PENDING`	Monte is about to speak	None
`SPEAKING`	Monte is talking	None, unless the worker talks over Monte
`HANGOVER`	200 ms after Monte stops	None

Talking over Monte (barge-in). While Monte speaks the session measures how loud the leftover echo is. If the microphone is clearly louder than that for about 220 ms, it stops Monte, sends the last 1.2 seconds of audio so the worker's first words are not lost, and returns to `OPEN`. The thresholds are the same ones `BargeInListener` already used.

While no audio is being sent, a `KeepAlive` message goes out every 4 seconds so Deepgram does not close the connection.

Losing the connection

If the WebSocket fails or closes, `useSpeech` logs it, switches to Vosk for the next answer, and tries to reconnect every 15 seconds. When the session comes back it takes the microphone over again.

Permissions

`useSpeech` asks for `RECORD_AUDIO` before opening the session. Previously only Vosk's own start call asked for it.

Understanding the answer

`handleUtterance` in `VoiceAgentFlow.js` decides what an answer means. The order below is the architecture; the step marked *not built* is skipped today.

1. **A list is being read out.** The answer goes to the read-out handler.
2. **The wake word alone** ("Monte"). Monte repeats the question.
3. **On-device parser** (`voiceParser.js`). Handles numbers, option numbers ("two", "el dos"), yes/no, go back, weights such as "15.5 kilos" or "quince coma cinco", and summary commands. No network.

4. **Jev option scoring** — *not built*. On an option screen, any answer that is not a plain number goes to Jev, which returns the closest option and a confidence score. See Option scoring with Jev. Until this exists, option names are matched by the on-device parser in step 3.
5. `interpret` **Cloud Function.** When nothing above could read the answer. Claude maps the transcript and the current screen's options to one canonical command. A result with confidence of 0.3 or more is acted on.
6. **Last resort.** The raw text goes to the screen's handler, which asks the worker to say the number or the name.

Today every option pick is confirmed aloud ("Confirming your selection: Torre 4. Shall I proceed?") before it is applied. With Jev, a pick at 0.9 confidence or more would skip that confirmation.

Cloud Functions

All in `functions/index.js`, region `us-central1`. API keys are read from `functions/.env` and never reach the app.

`interpret` — transcript to command

Input	`{ transcript, phase, options, language, context }`
Output	`{ command, confidence, note, model }`
Model	`claude-opus-5-5`, effort `low`, through `@anthropic-ai/sdk`
Output format	JSON schema enforced by the API, so the reply always parses
Client timeout	7 seconds (`llmInterpreter.js`)

Model choice was measured on a Spanish category answer: Claude Opus 5.5 at low effort took about 1.8 s, Claude Haiku 4.5 about 2.1 s. Fast mode is not available on the current Anthropic account (the API reports a fast-mode limit of zero).

Refusal fallback is on (`fallbacks: "default"`): if a safety check declines a request, the API re-runs it on another model. A refusal that survives that returns an empty command.

Local test results, real API:

Screen	Worker said	Command	Confidence
category	"eh lo que tengo acá son unas botellas de plástico y unos envases"	Plásticos	0.85
category	"cajas de cartón"	Papel Cartón	0.95
category	"esperá que me está sonando el teléfono"	*(none)*	0
location	"Torre $12.03"	Torres 1 2 3	0.7
weight	"quince no perdón diecisiete kilos"	seventeen kilos	0.9
addbags	"sí dale"	yes	0.95

Each call took between 1.6 and 2.9 seconds.

`transcribe` — recorded answer to text

Sends a recorded answer (16 kHz WAV, base64) to Deepgram `nova-3` with the screen's option names as key terms. Returns `{ transcript, command: "", confidence, model }`; `command` is always empty because Deepgram only transcribes.

Used only on the Vosk path, where the phone records the answer and sends it for a better transcript. While the live session is connected nothing calls it.

`deepgramToken` — short-lived token for the live session

Asks Deepgram for a 60-second token so a release build can open the WebSocket without carrying the API key. The token is only needed to connect; an open session outlives it. See Known issues: the current key cannot create tokens.

Configuration and secrets

File	Variable	Used by	In git
`functions/.env`	`DEEPGRAM_API_KEY`	`transcribe`, `deepgramToken`	No
`functions/.env`	`ANTHROPIC_API_KEY`	`interpret`	No
`functions/.env`	`TYPESAFE_API_KEY`	Jev `choose`, once built	No
`functions/.env`	`STT_MODEL` (optional, default `nova-3`)	`transcribe`	No
`functions/.env`	`APP_SHARED_TOKEN` (optional)	all three; must match `llmInterpreter.js`	No
`.env.local`	`EXPO_PUBLIC_DEEPGRAM_API_KEY`	the app, **development only**	No

The development key is bundled into the app. Any variable starting with `EXPO_PUBLIC_` is compiled into the JavaScript the phone runs, and can be read out of an APK. It exists so a development build can reach Deepgram while `deepgramToken` is not deployed. Before building anything that leaves your hands, delete `.env.local` (or that line) so the app uses `deepgramToken`.

All three API keys currently in use (Deepgram, Anthropic, TypeSafe) were pasted into a chat session. Rotate them before the project is shared more widely.

Gemini is no longer used. `LLM_API_KEY`, `LLM_BASE_URL` and `LLM_MODEL` have been removed.

Building and running on a phone

Android only: Vosk and the `voice-duplex` module have no iOS implementation.

```
npm install
./scripts/fetch-vosk-models.sh          # EN + ES models, about 125 MB, gitignored
export ANDROID_HOME=~/Library/Android/sdk
export JAVA_HOME=/Library/Java/JavaVirtualMachines/microsoft-17.jdk/Contents/Home   # JDK 17; 25 is
npx expo prebuild --platform android --clean
cd android && ./gradlew assembleDebug && cd ..

adb install -r android/app/build/outputs/apk/debug/app-debug.apk
adb reverse tcp:8081 tcp:8081
npx expo start --dev-client --port 8081
```

A first build takes about 10 minutes; later ones about 20 seconds. A change to any Kotlin file under `modules/voice-duplex/` needs a rebuild and reinstall. JavaScript changes only need the app reopened.

A debug build loads its JavaScript from the Metro server on the computer, so the phone has to stay on USB with Metro running.

Xiaomi / MIUI. In Developer options turn on *USB debugging*, *Install via USB*, and *USB debugging (Security settings)*. Without the second one `adb install` fails with `INSTALL_FAILED_USER_RESTRICTED`; retrying usually works once the on-phone prompt has been accepted. If `adb devices` shows nothing while the phone is plugged in, run `adb kill-server` and try again, and check the cable carries data.

Deploying the functions

```
cd functions && npm install && cd ..
firebase deploy --only functions        # project obra-padre-cacho, needs the Blaze plan
```

The Firebase CLI on the development machine is signed in to an account that cannot see the project, so this has not been run.

Diagnostics

Where	What it shows
Voice Agent → 🔧 Mic test	Runs Vosk directly with a live event log. Closes the live session while open.
`adb logcat -s VoiceDuplex`	Native session: `stream open`, `live mic open (aec true)`, `barge-in: speech … dB over Monte … dB`, `stream error …`.
Event log in the app	`live transcription connected`, `result (live): "…"`, `live transcription error: … — using offline voice`, `LLM → "…" (0.85)`.

A refused Deepgram connection reports Deepgram's own reason (its `dg-error` header) in the `stream error` line.

What has and has not been tested

Tested:

- Deepgram pre-recorded and live transcription from a computer, with synthesized English and Spanish clips.
- The live session on one phone, in English: connects, echo canceller active, barge-in triggers, transcript appears and drives the flow.
- `transcribe` and `interpret` run locally against the real APIs.

Not tested:

- Real workers, Spanish on the phone, or noisy sites.
- Any deployed Cloud Function, or the app calling one.
- `deepgramToken` end to end.
- Reconnection after a dropped connection on a phone.
- A full collection from first question to submit in one session.

Known issues

1. `smart_format` **mangles spoken digits.** "Torres one two three" came back as "Torre $12.03" and the wrong location was offered. To fix: turn `smart_format` off, or replace it with `numerals=true`, in `liveUrl`, then check weights still parse. Not yet done.
2. **Weak option matches are accepted.** `matchOption` scores candidates but returns only the winner, so a single shared word can select an option. The confirmation step catches it, at the cost of an extra exchange.
3. **The Deepgram key cannot create tokens.** `POST /v1/auth/grant` returns `FORBIDDEN: Insufficient permissions`. `deepgramToken` needs a key with the Member role or higher.
4. **Functions are not deployed**, so the phone has no cloud interpretation.
5. **One field per answer.** "Plástico, veinticuatro kilos" in one breath sets the category only; the weight is asked for separately.
6. **Billing.** The session streams whenever it is the worker's turn, silence included, and Deepgram bills streamed audio time.

Option scoring with Jev

Stage 3 of the architecture. **Not built yet**: there is no Jev code in the app or in `functions/`. Jev itself has been measured from a computer against the real option lists (results below). The key is in `functions/.env` as `TYPESAFE_API_KEY`.

Goal. On every option screen, send what the worker said to Jev and take the option it scores highest, with a confidence value. This applies whether the word was heard cleanly, misheard, or buried in a sentence.

Measured results

26 Spanish answers written for the test, sent to `jev-latest` (`jev-1.13.0`) with each screen's real options plus "none of these". One run, 7 October 2026.

Kind of answer	Example	Jev picked	Confidence
Clean	"Plásticos"	Plásticos	1.00
Clean	"la torre cuatro"	Torre 4	0.98
Misheard	"pla stick os"	Plásticos	0.81
Misheard	"mes clado"	Mezclado	0.69
Misheard	"bet natural"	PET Natural	0.97
Misheard	"nailon transparente"	Nylon Transparente	0.99
Misheard	"Torre $12.03"	Torres 1 2 3	0.30
Sentence	"eh lo que tengo acá son unas botellas de plástico y unos envases"	Plásticos	0.99
Sentence	"restos de comida y cáscaras"	Orgánico	1.00
Sentence	"está todo junto revuelto no se puede separar"	Mezclado	0.98
Sentence	"no sé qué es esto la verdad"	Sin Identificar	0.50
Off-topic	"esperá que me está sonando el teléfono"	none of these	0.99
Off-topic	"qué calor que hace hoy"	none of these	0.99
Ambiguous	"torre" (two towers on the list)	none of these	0.73

- **Accuracy:** 24 of the 25 answers that had one right option. The miss was "telgopor" → Poliestireno at 0.51, where the test expected Espuma; telgopor is expanded polystyrene, so that pick is defensible and its low confidence would have triggered a confirmation.
- **Speed:** median 103 ms, slowest 284 ms. Fast enough to run on every option answer.
- **Confidence tracked difficulty:** clean answers scored 0.98–1.00, garbled ones 0.3–0.8, and off-topic speech went to "none of these".

Limits of this test: the phrases were invented, not recorded from workers; it is a small sample; and it ran from a computer, not through the app.

Why Jev. TypeSafe's `choice` question takes a text and a fixed set of options and returns the chosen option, a probability for every option, and a confidence between 0 and 1. It cannot return an option that is not in the list.

```
POST https://api.typesafe.ai/v1/systemone
{
  "state": "eh tengo unas botellas de plástico",
  "model": "jev-latest",
  "questions": {
    "pick": {
      "type": "choice",
      "instructions": "Which waste category did the worker choose?",
      "criteria": { "paper": "Papel Cartón", "plastics": "Plásticos", "none": "None of these" }
    }
  }
}
```

Flow.

```
Deepgram text
   ▼
Jev choice   options = this screen's list + "none"
   ▼
confidence ≥ 0.9        select directly, no confirmation
0.5 – 0.9               select and ask the worker to confirm
below 0.5, or "none"    ask again
```

The 0.9 and 0.5 bands are TypeSafe's own suggested starting points and should be tuned on real answers.

Screens. Client, location, category, sub-category, and yes/no steps.

Kept outside Jev.

- Weights. A weight is any number, not a pick from a list, and Deepgram already writes the number into the text for `parseNumber` to read.
- "Number two" / "el dos". TypeSafe documents that Jev does not count reliably, so number picks map straight to the option.
- Offline. With no connection the on-device matcher is used.

Where it lives. A new Cloud Function, `choose`, called with plain `fetch` (TypeSafe publishes Python and JavaScript SDKs and a REST API; there is no Java package). The key stays on the server. On the app side, `llmInterpreter.js` calls it and `handleSelection` in `VoiceAgentFlow.js` applies the confidence bands.

Steps to build it.

1. ~~Add `TYPESAFE_API_KEY` to `functions/.env`.~~ Done.
2. ~~Measure Jev on the real option lists in Spanish.~~ Done, see above.
3. Add the `choose` Cloud Function.
4. Route option answers to it and apply the confidence bands.
5. Deploy and test on a phone with workers.

Open questions. Price and rate limits are not in TypeSafe's documentation. The confidence value reports how concentrated the probabilities are, not whether the pick is right, so the 0.9 and 0.5 bands still need checking against answers recorded from real workers.

Jev and Claude together. With Jev handling option picks, `interpret` remains for summary commands ("edit the weight for cartón", "delete plásticos") and messy weight corrections. It could be removed later if Jev proves sufficient.

Sources: introduction, quick start, confidence, known limits of jev-1.13.

File map

File	Role
`components/voice/VoiceAgentFlow.js`	The screen and conversation flow; decides when the session is on.
`components/voice/useSpeech.js`	Voice engine: live session, Vosk fallback, text-to-speech, barge-in.
`components/voice/deepgramStream.js`	Deepgram URL, authorization, transcript clean-up. **New.**
`components/voice/llmInterpreter.js`	Calls the Cloud Functions (`interpret` , `transcribe` , `deepgramToken`).
`components/voice/voiceParser.js`	On-device parser: numbers, yes/no, option matching, summary commands.
`modules/voice-duplex/…/StreamSession.kt`	Native mic → WebSocket session, echo gating, barge-in. **New.**
`modules/voice-duplex/…/VoiceDuplexModule.kt`	Native module surface: `startStream` , `stopStream` , `streamSpeakBegin` , `streamSpeakEnd` .
`modules/voice-duplex/…/CallSpeaker.kt`	Monte's voice on the call path; `hold()` keeps call audio on for the session.
`modules/voice-duplex/…/AnswerListener.kt` , `BargeInListener.kt`	Vosk path, unchanged.
`modules/voice-duplex/android/build.gradle`	Adds OkHttp 4.9.2 for the WebSocket.
`functions/index.js`	`interpret` (Claude), `transcribe` (Deepgram), `deepgramToken` . Jev's `choose` will be added here.
`scripts/fetch-vosk-models.sh`	Model download; fixed to run on macOS's bash 3.2.